

A Communication-Efficient Hierarchical Federated Learning Framework via Shaping Data Distribution at Edge

Yongheng Deng, Feng Lyu^{ID}, *Senior Member, IEEE*, Tengxi Xia, *Graduate Student Member, IEEE*, Yuezhi Zhou^{ID}, *Senior Member, IEEE*, Yaoyue Zhang, *Senior Member, IEEE*, Ju Ren^{ID}, *Senior Member, IEEE*, and Yuanyuan Yang^{ID}, *Life Fellow, IEEE*

Abstract—Federated learning (FL) enables collaborative model training over distributed computing nodes without sharing their privacy-sensitive raw data. However, in FL, iterative exchanges of model updates between distributed nodes and the cloud server can result in significant communication cost, especially when the data distributions at distributed nodes are imbalanced with requiring more rounds of iterations. In this paper, with our in-depth empirical studies, we disclose that extensive cloud aggregations can be avoided without compromising the learning accuracy if frequent aggregations can be enabled at edge network. To this end, we shed light on the hierarchical federated learning (HFL) framework, where a subset of distributed nodes can play as edge aggregators to support edge aggregations. Under the HFL framework, we formulate a communication cost minimization (CCM) problem to minimize the total communication cost required for model learning with a target accuracy by making decisions on edge aggregator selection and node-edge associations. Inspired by our data-driven insights that the potential of HFL lies in the data distribution at edge aggregators, we propose ShapeFL, i.e., **SH**aping **d**ata **d**ist**R**ibution at **E**dge, to transform and solve the CCM problem. In ShapeFL, we divide the original problem into two sub-problems to minimize the per-round communication cost

and maximize the data distribution diversity of edge aggregator data, respectively, and devise two light-weight algorithms to solve them accordingly. Extensive experiments are carried out based on several opened datasets and real-world network topologies, and the results demonstrate the efficacy of ShapeFL in terms of both learning accuracy and communication efficiency.

Index Terms—Hierarchical federated learning, communication efficiency, edge computing, distributed edge intelligence.

I. INTRODUCTION

DUE to the high penetration of mobile devices and advancements in sensing technology, massive amounts of data are continuously generated at edge [2], [3], [4]. Such vast data is usually aggregated and stored in the cloud, coupled with machine learning algorithms, to enable multifarious intelligent services [5], [6], [7], [8]. However, significant privacy leakage issue arises when sensitive raw data is delivered to the cloud via network transmission, which, as the uppermost impetuses, pushes forward the emergence of federated learning (FL) [9], [10], [11]. Unlike the centralized learning paradigm, FL enables collaborative learning of a shared global model at distributed computing nodes using their local raw data. Instead of delivering the raw data to the cloud, only the learned model updates are sent to the cloud server for aggregation. Then, the global model at the cloud server is updated and sent back to the distributed computing nodes for the next round of iteration. In this way, the model is learned without compromising user privacy.

In FL, one significant challenge is the prohibitive communication cost caused by the iterative exchanges of model updates between the distributed nodes and cloud server. Particularly, to achieve a desired learning accuracy, sufficient rounds of cloud aggregation are required especially when the local training data of distributed nodes are typically imbalanced (i.e., the training data has only a few labels, or the amount of data per label is skewed), which has been verified by our pilot experiments. In this regard, frequent model updates exchanges between distributed nodes and the cloud server is critical to optimize the global model. Contradictorily, distributed nodes are usually restrained in communication/bandwidth resources, which is prohibitive to communicate with the cloud frequently. Moreover, as the learned model structure is becoming much complex (e.g., deep neural networks), the transmitted data volume of model updates will increase significantly, which further aggravate the communication cost of FL systems.

Manuscript received 13 October 2022; revised 21 June 2023 and 26 November 2023; accepted 3 February 2024; approved by IEEE/ACM TRANSACTIONS ON NETWORKING Editor H. Shen. Date of publication 13 February 2024; date of current version 18 June 2024. This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFC2009805; in part by the National Natural Science Foundation of China under Grant 62320106006, Grant 62122095, Grant 62341201, and Grant 62072472; in part by 111 Project under Grant B18059; in part by the Young Talents Plan of Hunan Province of China under Grant 2021RC3004; in part by the Central South University Innovation-Driven Research Programme under Grant 2023CXQD029; and in part by the Guoqiang Institute, Tsinghua University. This work was accepted in part by the 41th IEEE International Conference on Distributed Computing Systems (IEEE ICDCS 2021) [DOI: 10.1109/ICDCS51616.2021.00012]. (Corresponding author: Feng Lyu.)

Yongheng Deng and Tengxi Xia are with the Department of Computer Science and Technology, Beijing National Research Center for Information Science and Technology (BNRist), Tsinghua University, Beijing 100084, China (e-mail: dyh2024@tsinghua.edu.cn; xtx19@mails.tsinghua.edu.cn).

Feng Lyu is with the School of Computer Science and Engineering, Central South University, Changsha 410083, China (e-mail: fenglyu@csu.edu.cn).

Yuezhi Zhou, Yaoyue Zhang, and Ju Ren are with the Department of Computer Science and Technology, Beijing National Research Center for Information Science and Technology (BNRist), Tsinghua University, Beijing 100084, China, and also with the Zhongguancun Laboratory, Beijing 100084, China (e-mail: zhouyz@tsinghua.edu.cn; zhangyx@tsinghua.edu.cn; renju@tsinghua.edu.cn).

Yuanyuan Yang is with the Department of Electrical and Computer Engineering, Stony Brook University, Stony Brook, NY 11794 USA (e-mail: yuanyuan.yang@stonybrook.edu).

Digital Object Identifier 10.1109/TNET.2024.3363916

1558-2566 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See <https://www.ieee.org/publications/rights/index.html> for more information.

Prior efforts in the literature to reduce the communication cost of FL mainly fall into three camps. (1) *Reducing the model update size*. In this regard, compression schemes, such as sparsification [12], [13], quantization [14], and sketching [15], [16], are employed to compress the model updates to reduce the size of messages communicated at each round of FL. (2) *Reducing the number of model update*. Previous works along with this direction performed model update selection [17], [18] or client selection [19] to reduce the number of model updates communicated to the cloud server. (3) *Reducing the required communication rounds*. This class of prior arts proposed several optimization algorithms for FL in data imbalance environments, e.g., FedProx [20], SCAFFOLD [21], to accelerate model convergence such that the communication rounds for reaching the target accuracy can be reduced.

Along a different avenue, this paper aims to reduce the communication cost and improve the learning accuracy of FL by exploiting the underlying potential of edge aggregations. Typically, to guarantee satisfying learning performance, as shown in Fig 1 (a), distributed nodes need to frequently commit the model updates to the aggregator located in the cloud, despite the relatively high communication cost posed to them, especially in the case that they have uneven data distributions. According to our pilot experiments, we find that *frequent cloud aggregations can be avoided without performance degradation, if model aggregations can be first performed at edge*. The learning performance can even be enhanced if the *edge aggregation* is properly configured. This inspires us to investigate a hierarchical federated learning (HFL) framework (Fig 1 (b)), where a subset of distributed nodes can be selected as *edge aggregators*, to communicate with nearby distributed nodes for edge model aggregations with low communication cost. After several rounds of edge aggregations, edge aggregators then commit the edge model updates to the cloud for global aggregation. The merit lies in that frequent edge aggregations can be enabled with an acceptable communication cost, avoiding frequent communications to the cloud. Besides, the proposed framework is orthogonal to the aforementioned prior studies, and hence, the proposed methods in this paper can be seamlessly integrated with existing approaches, e.g., compression techniques, to collectively reduce the communication cost of FL. In addition to reducing the communication cost per round of aggregation, with in-depth empirical studies, we also disclose another significant potential of the HFL framework in accelerating model convergence and improving the learning accuracy. That is, *if the data distribution can be tuned with a balanced shape at edge aggregators via ingenious node-edge association, the learning accuracy can be improved and the cloud aggregations can be further reduced*.

Based on these data-driven insights, we propose ShapeFL, i.e., SHaping dAtA distRibution at Edge, to minimize the communication cost required to train the global model with reaching a target accuracy in the proposed HFL framework. Toward this, ShapeFL follows two optimization directions. On the one hand, we focus on reducing the communication cost for the exchange of model updates among distributed nodes, edge aggregators and the cloud aggregator, in order to

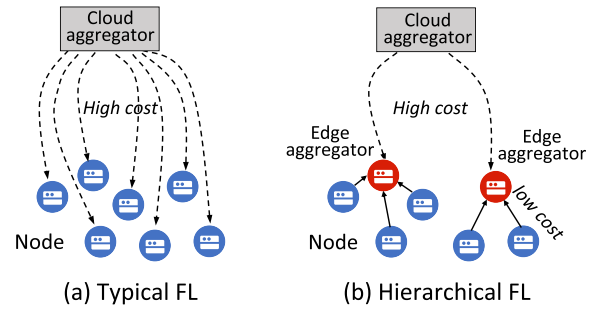


Fig. 1. Typical cloud-based FL vs. hierarchical FL.

reduce the per-round communication cost. On the other hand, we aim to shape the data distribution at edge aggregators to make their data distributions being more balanced, which can *reduce the communication rounds required* to reach the target model accuracy. Since the two goals cannot be achieved simultaneously, in *ShapeFL*, we devise two lightweight algorithms to make fine trade-offs between them by making decisions on the edge aggregator selection and node-edge association in order to minimize the total communication cost. To evaluate the performance of ShapeFL, we build practical HFL systems by adopting the real-world network topologies and general learning tasks. Extensive experiments are conducted under various settings, and the results demonstrate that ShapeFL can enhance the learning accuracy and reduce the communication cost significantly, in comparison with competitive benchmarks.

We highlight our major contributions as follows.

- We disclose valuable insights with pilot experiments: 1) applying the HFL framework is promising in achieving low-cost and high-performance learning; and 2) shaping data distributions at edge can facilitate enhanced model accuracy and further reduce communication cost, which are valuable for researchers to optimize the desired FL system.
- We investigate a flexible HFL framework and propose ShapeFL to optimize the communication cost with lightweight edge aggregator selection and node-to-edge association strategies, which contributes to a communication-efficient FL.
- We implement ShapeFL prototypes and extensive experimental results demonstrate that ShapeFL can be readily applied to various network topologies and different learning tasks with superior learning performance.

The remainder of this paper is organized as follows. We shed light on our motivation in Section II. In Section III, we present the system description and define the research problem. Section IV elaborates on the design of ShapeFL. We conduct extensive experiments in Section V, Section VI discusses the limitation of this work, and the related work is reviewed in Section VII. Finally, we conclude this paper and direct our future work in Section VIII.

II. MOTIVATION

In this section, we carry out pilot experiments to highlight the motivations for developing the HFL framework and the design of ShapeFL.

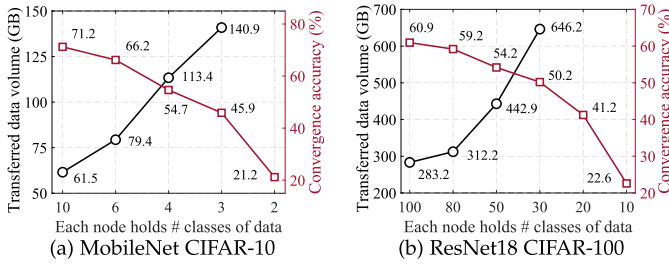


Fig. 2. Transferred data volume and convergence accuracy of FL under imbalanced training data.

A. Communication Overhead in FL

FL is an iterative learning process, where distributed nodes have to continuously commit the model updates to the server for aggregation to guarantee the learning accuracy. Generally, achieving a desired model accuracy requires considerable rounds of aggregation, which results in significant communication overhead. Besides, the training data generated at distributed nodes are typically imbalanced with skewed label distributions due to spatio-temporal behavior preferences. Such data imbalance will not only degrade the learning accuracy but also impede the model convergence speed, resulting in more communication rounds required to achieve the desired model accuracy with exacerbating the communication overhead. To empirically verify this, we build an FL system with 30 nodes and a cloud server, and then measure the communication overhead and learning accuracy when varying the class of data held by each node (The detailed experimental setup is shown in Section V-A). Fig. 2 shows the convergence accuracy of the MobileNet and ResNet18 models trained with the CIFAR-10 and CIFAR-100 datasets respectively, as well as the total transferred data volume required to achieve 50% model accuracy. We can observe that, as the data imbalance degree increases, i.e., the training data of each node with fewer label classes, the achieved model accuracy decreases drastically while the transferred data volume increases significantly. Taking the ResNet18 model as an example, when each node holds 10 label classes of CIFAR-100 training data samples, the model accuracy can only reach 22.6% while the total transferred data volume of nodes can reach much up to 646.2 GB (the learning accuracy cannot reach the target value in this setting). Besides, it is worth noting that the model structures of MobileNet and ResNet18 are relatively lightweight, when applying more complex deep learning models, the transferred data volume will become larger, leading to prohibitive communication overhead for FL systems. Therefore, there is an immediate need to improve the learning accuracy and reduce the communication cost for FL, especially when distributed nodes have imbalanced data.

B. Potential of Using the HFL Framework

To improve the learning accuracy with imbalanced training data, increasing the model aggregation frequency is viable. To demonstrate this, we evaluate the performance of the ResNet18 model trained with the CIFAR-100 dataset under different aggregation frequencies. Here, the training data of the 30 nodes is set to be imbalanced, each having only 20 classes of samples. Fig. 3 shows the learning results, where $AF =$

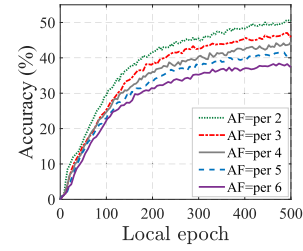


Fig. 3. Impact of aggregation frequency in FL.

per i represents that each node commits their model updates to the cloud server to execute one aggregation every i local epochs of training. Hence, a smaller i means more frequent model aggregation. It can be observed that the achieved model accuracy of FL can increase with the aggregation frequency. It indicates that when distributed nodes have imbalanced training data, to guarantee the learning performance, frequent model aggregations are a requisite, entailing frequent communications between distributed nodes and the cloud server with more communication costs in FL.

It would be tempting if some aggregations can be conducted at the network edge since the communication cost to the edge is negligible compared with that to the cloud. Therefore, among the 30 nodes, we choose 3 nodes as additional edge aggregators, each conducting model aggregations for random 10 nodes, and then the edge aggregators commit the edge model updates to the cloud server for global aggregation. In this way, the FL system evolves to a hierarchical architecture. To explore the potential of HFL framework, we fix the cloud aggregation frequency as 5 and vary the edge aggregation frequency to evaluate the learning performance of HFL. From the results in Fig. 4, we can make two major observations. First, compared to the cloud-based FL, HFL can reach comparable or even better learning performance with fewer cloud aggregations. For example, when the cloud aggregation frequency (CAF) is set to per 5, 500 local epochs of training requires 100 cloud aggregations in the cloud-based FL, while only 50 cloud aggregations are required in HFL when the edge aggregation frequency (EAF) is set to per 2. Meanwhile, HFL can also outperform CFL significantly in learning accuracy. Second, it can be observed that increasing the edge aggregation frequency can improve the learning accuracy. Fortunately, although frequent aggregations are prohibitive in the cloud-based FL due to the expensive node-to-cloud communication cost, frequent edge aggregations can be enabled with an acceptable communication cost to improve the learning accuracy. The results demonstrate the significant potential of using the HFL framework, which can not only reduce the communication cost by reducing the cloud aggregations but also enhance the FL performance by enabling more frequent aggregations at edge networks.

C. Trick of Using the HFL Framework

We further investigate the trick of utilizing the HFL framework. To ensure the communication efficiency of HFL, frequent cloud aggregations are preferably avoided. We have disclosed in Fig. 3 that the aggregation frequency can affect the learning accuracy when the training data of nodes are

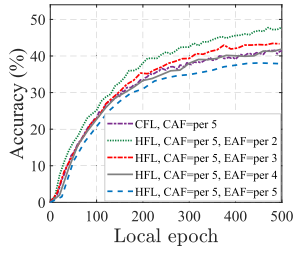


Fig. 4. Impact of edge aggregation frequencies.

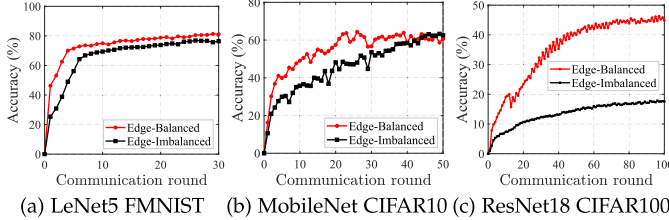


Fig. 5. Performance under edge-balanced and edge-imbalanced settings.

imbalanced. However, we experiment and find that the aggregation frequency has a negligible effect on balanced training data. Therefore, we speculate that if the data distribution of edge aggregators¹ can be shaped to be balanced, even infrequent cloud aggregations should be less disruptive to the performance of HFL. In other words, the cloud aggregations can be further reduced with negligible accuracy penalty by shaping the data distribution of edge aggregators to be balanced. To verify this, we first set the edge and cloud aggregation frequency of the HFL system as per 1 and per 10 respectively, and compare the HFL performance between *Edge-Balanced* and *Edge-Imbalanced* settings, representing a balanced and imbalanced data distribution at the 3 edge aggregators, respectively. Specifically, in *Edge-Balanced* setting, we assign each edge aggregator 10 nodes with different classes of training data to make the data distribution of each edge aggregator close to be uniform, while in *Edge-Imbalanced* setting, we assign each edge aggregator 10 nodes with the same class of training data as much as possible, each having a rather skewed data distribution. As shown in Fig. 5, we can observe that, for the LeNet5 FMNIST, MobileNet CIFAR-10, and ResNet18 CIFAR-100 learning tasks, the performance under the *Edge-Balanced* setting can outperform that under the *Edge-Imbalanced* case significantly in terms of both model accuracy and convergence speed. Therefore, we can conclude that in the HFL framework, when the edge and cloud aggregation frequencies are fixed, shaping the data distribution at edge aggregator can affect the FL performance significantly.

Then, taking the ResNet18 CIFAR-100 task as an demonstration, we fix the edge aggregation frequency as per 1 and examine the performance of HFL under *Edge-Balanced* and *Edge-Imbalanced* cases by varying cloud aggregation frequencies. From the results in Fig. 6, we can observe that the decrease of the cloud aggregation frequency leads to a sharp performance drop in terms of the model learning accuracy and convergence speed under the *Edge-Imbalanced* case, but has negligible impact on performance in the *Edge-Balanced* case. It means that if the data distribution of edge aggregators

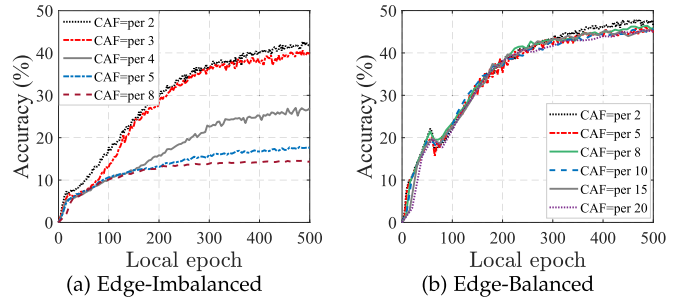


Fig. 6. Performance of edge-imbalanced and edge-balanced cases by varying cloud aggregation frequencies.

can be shaped to be balanced, frequent cloud aggregations can be avoided without performance degradation. Therefore, we can conclude that shaping balanced data distributions at edge aggregators via delicate node-edge association can not only improve the learning accuracy, but also has great potential in reducing the communication cost of HFL systems.

III. SYSTEM DESCRIPTION AND PROBLEM DEFINITION

A. Hierarchical Federated Learning System

We consider a distributed FL system located within a broad area, where there are one cloud server and a set of N distributed computing nodes (denoted by $\mathcal{N}$). Each distributed node has imbalanced training data stored locally, which is practical in real-world scenarios since the personal data of distributed nodes are generally location- and device-related. A subset of distributed computing nodes in $\mathcal{N}$ can be selected as edge aggregators to perform model aggregation at edge if necessary, denoted by $\mathcal{N}_e \subset \mathcal{N}$. As shown in Fig. 7, distributed computing nodes, edge aggregators, and the cloud server collectively constitute the HFL system. In specific, during each round of cloud aggregation, the edge aggregators first download the shared global model from the cloud server. Then, each distributed computing node downloads the model from its associated edge aggregator, and trains the model locally with their own data samples. After κ_e local training epochs, each node will commit the local model updates to its associated edge aggregator. Edge aggregators conduct the model aggregations and send back the aggregated model updates to associated nodes. After κ_c edge aggregations, each edge aggregator will commit edge model updates to the cloud server for global model aggregations. Afterwards, the cloud aggregator performs aggregation and send back the updated global model to the edge aggregators for the next round of model training. The above process will repeat until the shared global model reaches a target learning accuracy. Table I shows the key notations appeared in this paper.

B. Problem Definition

Under the proposed HFL framework, we aim to minimize the communication cost raised by the edge and cloud aggregations to achieve a target learning accuracy. Therefore, we cast the following communication minimization problem.

Definition 1 (Communication Cost Minimization (CCM)): Given a set of distributed computing nodes $\mathcal{N}$ and the set of candidate edge aggregators $\mathcal{N}_e$, how to select a subset of edge aggregators and how to associate the computing

¹The data distribution of an edge aggregator represents the data distribution of the training data set of nodes associated with it.

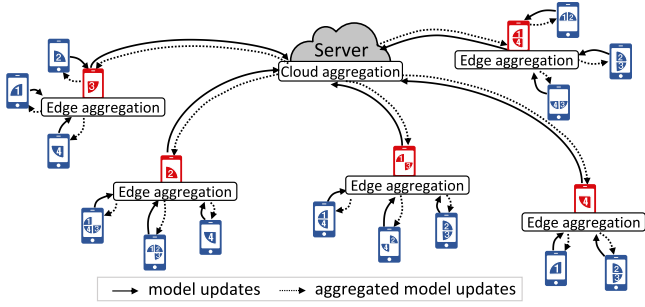


Fig. 7. An overview of the HFL framework.

TABLE I
KEY NOTATIONS

Notation	Definition
$\mathcal{N}$	The set of distributed computing nodes
$\mathcal{N}_e$	The set of nodes that can be selected as edge aggregators (candidate edge aggregators)
κ_e	The number of local epochs per edge aggregation
κ_c	The number of edge aggregations (edge epochs) per cloud aggregation (communication round)
κ	The number of communication rounds required to reach the target accuracy
x_e	Indicator of whether candidate edge aggregator node $e \in \mathcal{N}_e$ is selected as an edge aggregator
y_{ne}	Indicator of whether node n is associated with edge aggregator e
c_{ne}	The communication cost between node n and node e
c_{ec}	The communication cost between node e and the cloud server
B_e	The maximum number of nodes that can associate to edge aggregator e
$\mathcal{M}_e$	The set of nodes associated to edge aggregator e
$\mathcal{E}$	The set of the selected edge aggregators
D_n	The local training data size of node n
S_{ij}	The data distribution diversity between node i and node j
w_i	The local model updates of node i
w'_i	The local model updates of the linear layer of node i

nodes with the selected edge aggregators, such that the total communication cost between computing nodes, edge aggregators, and the cloud aggregator required for model training to reach a target accuracy is minimized?

Denote $x_e \in \{0, 1\}$ as a binary variable that indicates whether candidate edge node $e \in \mathcal{N}_e$ is selected as an edge aggregator ($x_e = 1$) or not ($x_e = 0$). Let the binary variable $y_{ne} \in \{0, 1\}$ indicate whether node $n \in \mathcal{N}$ is associated with edge aggregator e ($y_{ne} = 1$) or not ($y_{ne} = 0$). Denote by c_{ne} the communication cost of node n committing the local model updates to its associated edge aggregator e . Define κ as the total number of cloud aggregations performed at the cloud aggregator to reach a target learning accuracy. The total communication cost between distributed computing nodes and the edge aggregators can be represented by:

$$C_{ne}(\mathbf{Y}, \kappa) = \kappa \kappa_c \sum_{n \in \mathcal{N}} \sum_{e \in \mathcal{N}_e} y_{ne} c_{ne}, \quad (1)$$

where $\mathbf{Y} = \{y_{ne}\}_{n \in \mathcal{N}, e \in \mathcal{N}_e}$ are the node-edge association results. Likewise, define c_{ec} as the communication cost of edge aggregator e committing the edge model updates to the cloud aggregator. Then, the total communication cost between edge

aggregators and the cloud aggregator can be represented as follows:

$$C_{ec}(\mathbf{X}, \kappa) = \kappa \sum_{e \in \mathcal{N}_e} x_e c_{ec}, \quad (2)$$

where $\mathbf{X} = \{x_e\}_{e \in \mathcal{N}_e}$ are the edge aggregator selection results. The CCM problem can be formally formulated as

$$\min_{\mathbf{X}, \mathbf{Y}} C_{ne}(\mathbf{Y}, \kappa) + C_{ec}(\mathbf{X}, \kappa), \quad (3)$$

$$\kappa = f(\mathbf{X}, \mathbf{Y}) \quad (4)$$

$$\text{s.t. } x_e = 0, \quad \forall e \notin \mathcal{N}_e, \quad (5)$$

$$\sum_{e \in \mathcal{N}_e} y_{ne} = 1, \quad \forall n \in \mathcal{N}, \quad (6)$$

$$y_{ne} \leq x_e, \quad \forall n \in \mathcal{N}, \quad \forall e \in \mathcal{N}_e, \quad (7)$$

$$\sum_{n \in \mathcal{N}} y_{ne} \leq B_e, \quad \forall e \in \mathcal{N}_e, \quad (8)$$

$$x_e \in \{0, 1\}, \quad \forall e \in \mathcal{N}_e, \quad (9)$$

$$y_{ne} \in \{0, 1\}, \quad \forall n \in \mathcal{N}, \quad \forall e \in \mathcal{N}_e. \quad (10)$$

The constraint (5) limits that only the nodes that can play the role of edge aggregator can be selected as edge aggregators. The constraint (6) represents that each computing node has to be associated with one and only one edge aggregator. The constraint (7) requires that the candidate edge node e must be selected as an edge aggregator in order to enable the association from node n . Since distributed nodes usually have restrained communication and computation resources, we limit the number of computing nodes that can be associated to edge aggregator e within B_e , shown in constraint (8).

Notably, the decisions on $\mathbf{X}$ and $\mathbf{Y}$ can intangibly affect the value of κ , and we use $f(\cdot)$ to denote their relationship in Eq. (4). However, the factors influencing the convergence speed of a model are diverse, and the complex interdependencies among them are challenging to capture with a specific model $f(\cdot)$. This complexity makes it intractable to accurately model and optimize the CCM problem. Therefore, we resort to designing heuristic approaches to address the CCM problem. Based on the data-driven insights in Section II, in what follows, we consider the data distribution at edge aggregators to pinpoint the impact of κ , and then transform the original CCM problem into sub-problems and devise efficient algorithms to solve them.

IV. DESIGN OF SHAPEFL

In this section, we propose the design of the optimization framework ShapeFL to solve the CCM problem.

A. Overview of ShapeFL

Figure 8 shows the design of ShapeFL, which involves the components of problem transformation and algorithm design. For problem transformation, there are two optimization directions for solving the CCM problem, i.e., minimizing the communication cost in each round of cloud aggregation, and reducing the number of cloud aggregations required. Here, we first divide the CCM problem into two sub-problems along the two directions, where the sub-problem of per round communication cost minimization (PCCM) problem is

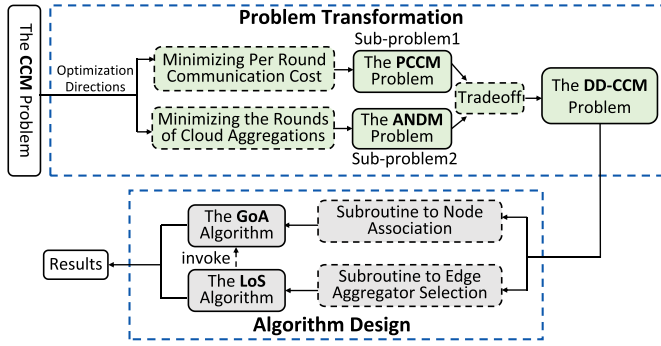


Fig. 8. The design of ShapeFL.

defined to minimize the communication cost per round of cloud aggregation. The sub-problem2 aims at making the data distribution of each edge aggregator more balanced. However, since the local training data distribution of each computing node is not available for privacy concerns, we try to associate the computing nodes with large differences in data distribution to the same edge aggregator to shape balanced edge data distributions. Therefore, we define the sub-problem2 as the associated node diversity maximization (ANDM) problem. By balancing the trade-offs between the two sub-problems, the original CCM problem is then transformed into a data distribution aware communication cost minimization (DD-CCM) problem. Due to the NP-hardness of the DD-CCM problem, we finally design two light-weight algorithms to determine the edge aggregator selection and node-edge association.

B. Problem Transformation

Sub-problem1 (PCCM). In each round of cloud aggregation, the objective is to minimize the total communication cost between distributed computing nodes, edge aggregators, and the cloud aggregator by making decisions on $\mathbf{X}$ and $\mathbf{Y}$. That is,

$$\min_{\mathbf{X}, \mathbf{Y}} J_c(\mathbf{X}, \mathbf{Y}) = \kappa_c \sum_{n \in \mathcal{N}} \sum_{e \in \mathcal{N}_e} y_{ne} c_{ne} + \sum_{e \in \mathcal{N}_e} x_e c_{ec},$$

s.t. constraints : (5), (6), (7), (8), (9), (10). (11)

Denote by $P_n = P(\mathbf{D}_n)$ the data distribution of node n , and denote by $P_e = P(\cup_{n \in \mathcal{M}_e} \mathbf{D}_n)$ the data distribution of edge aggregator e , where $\mathcal{M}_e$ is the set of nodes associated to edge aggregator e . Note that, since privacy concerns have ruled out any exposure of the raw data on distributed nodes, the data distribution P_n of each node is invisible and cannot be calculated for shaping balanced edge distributions. Fortunately, although the precise data distribution of each node is unknown, it is possible to estimate the data distribution similarity among computing nodes based on their committed model updates [22], [23]. Therefore, to make the data distribution of each edge aggregator more balanced, we advocate to associate the distributed nodes with dissimilar data distributions to the same edge aggregator as much as possible to complement the imbalanced data. Toward this, we define the sub-problem2 as follows.

Sub-problem2 (ANDM): For each edge aggregator, the objective is to maximize the data distribution diversity of the

associated computing nodes by making decisions on $\mathbf{X}$ and $\mathbf{Y}$. It can be formulated as follows.

$$\max_{\mathbf{X}, \mathbf{Y}} J_d(\mathbf{X}, \mathbf{Y}) = \frac{1}{|\mathcal{E}|} \sum_{e \in \mathcal{E}} \frac{1}{\binom{D_e}{2}} \sum_{i, j \in \mathcal{M}_e} S_{ij} D_i D_j,$$

s.t. constraints : (5), (6), (7), (8), (9), (10), (12)

where $\mathcal{E}$ is the set of the selected edge aggregators (i.e., $x_e = 1, \forall e \in \mathcal{E}$), D_i is the amount of training data size of computing node i , and $D_e = \sum_{n \in \mathcal{M}_e} D_n$ is the sum of the training data size of the nodes associated with edge aggregator e . S_{ij} represents the training data distribution diversity between node i and node j . Here, S_{ij} is calculated using the cosine distance between the model updates of the linear layer submitted by the nodes. Besides, in Eq. (12), the training data size is taken into account to make the data distribution of each edge aggregator more balanced.

It should be noted that, the optimal results of *Sub-problem1* and *Sub-problem2* cannot be always achieved simultaneously due to the competing objectives. The following tradeoff plays: should we associate nodes to edge aggregators according to the communication cost or the data distribution at edge aggregators? To balance the tradeoff, we incorporate a parameter γ to tune the weight of communication cost and balanced data distribution. Then, the original CCM problem can be transformed into the following DD-CCM problem:

$$\min_{\mathbf{X}, \mathbf{Y}} J_c(\mathbf{X}, \mathbf{Y}) - \gamma J_d(\mathbf{X}, \mathbf{Y}),$$

s.t. constraints : (5), (6), (7), (8), (9), (10). (13)

Challenges: To solve the DD-CCM problem, we face the following challenges. First, we have to decide how many and which edge aggregators should be selected, which however is non-trivial. On the one hand, if we choose more edge aggregators, although the communication cost between distributed computing nodes and edge aggregators can be reduced, the communication cost between edge aggregators and the cloud aggregator will increase. On the other hand, the edge aggregators should be closer to distributed computing nodes to decrease the node-edge communication cost, but should also be closer to the cloud aggregator to save the edge-cloud communication cost, conflicting with each other. Second, how to associate the distributed computing nodes with the edge aggregators also matters. On the one hand, we can reduce the communication cost per round by associating each node with the according nearest edge aggregator. On the other hand, for learning performance, we have to consider the data distribution at edge aggregators, while most of the time they cannot be achieved simultaneously. In fact, even if we disregard the data distribution at edge aggregators, the above problem is still NP-hard, which can be a polynomial reduction from the classic NP-hard *Capacitated Facility Location Problem* [24].

C. Algorithm Design

To address the above challenges, we devise two light-weight algorithms to optimize the edge aggregator selection and node-edge association decisions, respectively. Particularly, we first answer the following question: given a set of edge aggregators,

Algorithm 1 The Workflow of *GoA* Algorithm

Input : (1) edge aggregators set $\mathcal{E}$; (2) cost c_{ne} ; (3) data distribution diversity S_{ij} .

Output: the node association results $\mathbf{Y} = \{y_{ne}\}$.

```

1 Initialize  $\mathcal{M}_e \leftarrow \emptyset$ ,  $D_e \leftarrow 0$  foreach  $e \in \mathcal{E}$ ;
2 Initialize  $\mathcal{N}_a \leftarrow \mathcal{N}$ ;
3 repeat
4   foreach  $n \in \mathcal{N}_a$  do
5     foreach  $e \in \mathcal{E}$  and  $|\mathcal{M}_e| < B_e$  do
6        $\mathcal{M}'_e \leftarrow \mathcal{M}_e + \{n\}$ ;
7        $D'_e \leftarrow D_e + D_n$ ;
8        $\Delta S_{ne} \leftarrow \frac{\sum_{i,j \in \mathcal{M}'_e} S_{ij} D_i D_j}{\binom{D'_e}{2}} - \frac{\sum_{i,j \in \mathcal{M}_e} S_{ij} D_i D_j}{\binom{D_e}{2}}$ ;
9        $\Delta J_{ne} \leftarrow \kappa_c c_{ne} - \gamma \frac{1}{|\mathcal{E}|} \Delta S_{ne}$ ;
10    Find the node  $n$  and the edge aggregator  $e$  with the
        minimum  $\Delta J_{ne}$ ;
11     $\mathcal{M}_e \leftarrow \mathcal{M}_e + \{n\}$ ;
12     $\mathcal{N}_a \leftarrow \mathcal{N}_a - \{n\}$ ;
13     $D_e \leftarrow D_e + D_n$ ;
14     $y_{ne} \leftarrow 1$ ;
15 until  $\mathcal{N}_a = \emptyset$ 
16 return  $\{y_{ne}\}$ ;
```

how to associate distributed computing nodes to them, such that the communication cost and the edge data distribution can be jointly optimized without violating the constraints? To this end, we propose a Greedy-based nOde Association (*GoA*) algorithm to make decisions on node-to-edge associations. Based on *GoA*, we then employ a Local Search (*LoS*) algorithm to make decisions on the edge aggregator selection reversely.

Subroutine to Distributed Node Association. Given a set of edge aggregators $\mathcal{E}$, we need to determine which edge aggregator that each distributed node should associate to. It can be formulated as:

$$\min_{\mathbf{Y}} \kappa_c \sum_{n \in \mathcal{N}} \sum_{e \in \mathcal{E}} y_{ne} c_{ne} - \gamma \frac{1}{|\mathcal{E}|} \sum_{e \in \mathcal{E}} \frac{1}{\binom{D_e}{2}} \sum_{i,j \in \mathcal{M}_e} S_{ij} D_i D_j, \quad (14)$$

$$s.t. \ y_{ne} = 0, \quad \forall n \in \mathcal{N}, \forall e \notin \mathcal{E}, \quad (15)$$

$$\sum_{e \in \mathcal{E}} y_{ne} = 1, \quad \forall n \in \mathcal{N}, \quad (16)$$

$$\sum_{n \in \mathcal{N}} y_{ne} \leq B_e, \quad \forall e \in \mathcal{E}, \quad (17)$$

$$y_{ne} \in \{0, 1\}, \quad \forall n \in \mathcal{N}, e \in \mathcal{E}. \quad (18)$$

To solve the above distributed node association problem, we devise the *GoA* algorithm as shown in Algorithm 1. Specifically, the algorithm traverses all unassociated nodes and available edge aggregators with $|\mathcal{M}_e| < B_e$, and calculates the values of $\Delta J_{ne} = \kappa_c c_{ne} - \gamma \frac{1}{|\mathcal{E}|} \Delta S_{ne}$. The first term of ΔJ_{ne} represents the communication cost between node n and edge aggregator e in each cloud aggregation. The second term of ΔJ_{ne} is the average reduction of each aggregator in data distribution diversity between associated nodes after associating node n with edge aggregator e . Therefore, ΔJ_{ne} accounts for the change in communication cost and edge data distribution that it brings to the system when associating n with e . Based on the calculated values of ΔJ_{ne} , we identify

Algorithm 2 The Workflow of *LoS* Algorithm

```

1 Initialization: Feasible solution  $\mathcal{E}_s$ ;
2 repeat
3   'open' operation
4   foreach  $e \in \mathcal{N}_e - \mathcal{E}_s$  do
5     Compute  $J(\mathcal{E}_s + \{e\})$ ;
6     if  $J(\mathcal{E}_s + \{e\}) < J(\mathcal{E}_s)$  then
7        $\mathcal{E}_s \leftarrow \mathcal{E}_s + \{e\}$ ;
8       break
9   'close' operation
10  foreach  $e \in \mathcal{E}_s$  do
11    Compute  $J(\mathcal{E}_s - \{e\})$ ;
12    if  $J(\mathcal{E}_s - \{e\}) < J(\mathcal{E}_s)$  then
13       $\mathcal{E}_s \leftarrow \mathcal{E}_s - \{e\}$ ;
14      break
15  'swap' operation
16  foreach  $e \in \mathcal{N}_e - \mathcal{E}_s$  do
17    foreach  $e' \in \mathcal{E}_s$  do
18      Compute  $J(\mathcal{E}_s + \{e\} - \{e'\})$ ;
19      if  $J(\mathcal{E}_s + \{e\} - \{e'\}) < J(\mathcal{E}_s)$  then
20         $\mathcal{E}_s \leftarrow \mathcal{E}_s + \{e\} - \{e'\}$ ;
21        break
22 until  $T$  times or no operation can reduce the value of  $J$ 
23 return  $\mathcal{E}_s$ ;
```

the association pair of the node n and the edge aggregator e with the smallest value of ΔJ_{ne} , and then associate them. The above process repeats until all nodes are associated with one edge aggregator in $\mathcal{E}$ and finally the algorithm returns the node-edge association results.

Subroutine to Edge Aggregator Selection. With the node-edge association algorithm, we then focus on the edge aggregator selection problem to determine the set of edge aggregators. It is prone to confirm that the edge aggregator selection problem is a combinatorial problem with $O(2^{|\mathcal{N}_e|})$ possible options. To enable a time-efficient strategy, we devise a local search algorithm, i.e., *LoS*, for edge aggregator selection. Define $J_m(\mathcal{E}_s)$ as the optimal value of Eq. (14) with the given edge aggregator set $\mathcal{E}_s$. Additionally, if $\mathcal{E}_s$ violates the constraints (15)-(18), we set $J_m(\mathcal{E}_s) = +\infty$. Then,

$$J(\mathcal{E}_s) = J_m(\mathcal{E}_s) + \sum_{e \in \mathcal{E}_s} c_{ec}, \quad (19)$$

is the optimal value of Eq. (13) with the given $\mathcal{E}_s$. To optimize the value of $J(\mathcal{E}_s)$, we employ three types of operations [24], [25]. As shown in Algorithm 2, starting with a randomly selected set of edge aggregators $\mathcal{E}_s$ as the initial feasible solution, the algorithm repeats the following three local improvement operations until no operation can reduce the value of J .

open (e): We randomly choose a candidate edge aggregator e that is not in the current solution $\mathcal{E}_s$ and perform Algorithm 1 to calculate $J(\mathcal{E}_s + \{e\})$. If there exists a candidate edge aggregator e that can make $J(\mathcal{E}_s + \{e\}) < J(\mathcal{E}_s)$, we include it into the current solution $\mathcal{E}_s$.

close (e): We randomly choose an edge aggregator e in the current solution $\mathcal{E}_s$ and perform Algorithm 1 to calculate $J(\mathcal{E}_s - \{e\})$. If there exists such e that can make $J(\mathcal{E}_s - \{e\}) < J(\mathcal{E}_s)$, we remove it from the current solution $\mathcal{E}_s$.

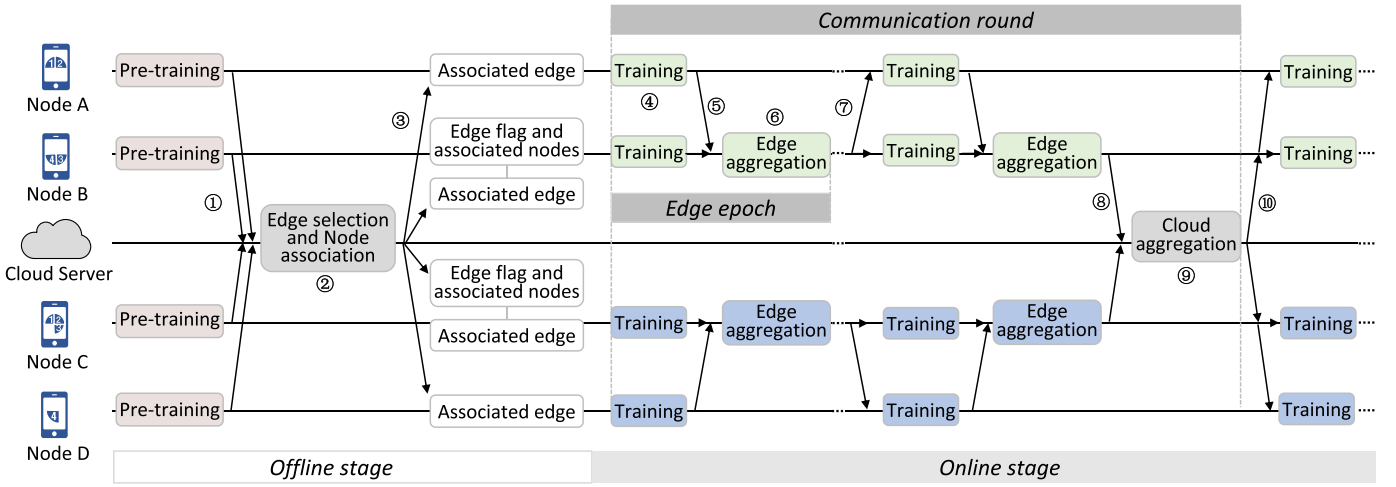


Fig. 9. The workflow of ShapeFL.

swap (e, e'): We randomly choose a candidate edge aggregator e that is not in the current solution $\mathcal{E}_s$, and an edge aggregator e' in the current solution $\mathcal{E}_s$. Then, we perform Algorithm 1 to calculate $J(\mathcal{E}_s + \{e\} - \{e'\})$. If there exists such a pair of e and e' that can get $J(\mathcal{E}_s + \{e\} - \{e'\}) < J(\mathcal{E}_s)$, we include e into the current solution $\mathcal{E}_s$ and remove e' from $\mathcal{E}_s$ as well.

D. Workflow of ShapeFL

Figure 9 illustrates the workflow of ShapeFL, including an offline stage and an online stage, details are shown in Algorithm 3. In the offline stage, distributed computing nodes first parallelly download the initialized global model from the cloud server and perform pre-training using their local training data. Then, the training data size and the model updates of the linear layer are committed to the cloud server (①). Utilizing these committed information of nodes as well as the communication costs between nodes,² the cloud server calculates the data distribution diversity among nodes and executes Algorithm 2 and Algorithm 1 to select edge aggregators and obtain the node-edge association results (②). Then, the cloud server sends a flag signal to the nodes that are selected as edge aggregators and sends the associated edge to each node (③). In the online stage, for each communication round, the cloud server performs one cloud aggregation after κ_c edge epochs. Specifically, for each edge epoch, each node parallelly performs κ_e epochs of local training (④) and commits the model updates to its associated edge aggregator (⑤). Then, each edge aggregator aggregates the received model updates (⑥) and sends back the aggregated model updates to their associated nodes (⑦) to start the next edge epoch. After κ_c edge epochs, each edge aggregator commits their aggregated model updates to the cloud server (⑧), which performs cloud aggregation (⑨) and sends the aggregated model updates back to each edge (⑩) for the next communication round. After κ communication rounds, the cloud server acquires an updated global model with achieving the target accuracy.

Algorithm 3 The Workflow of ShapeFL

```

/* Cloud server's process */
1 Initialize the global model with  $w(0)$ ;
2 foreach node  $n \in \mathcal{N}$  in parallel do
3    $(D_n, w_n^l) \leftarrow \text{PreTrain}(n, \kappa_p)$ ;
4    $\Delta w_n^l \leftarrow w_n^l - w(0)^l$ ;
5 Calculate  $S_{ij} \leftarrow \text{dist}(\Delta w_i^l, \Delta w_j^l)$  foreach  $i, j \in \mathcal{N}$ ;
6 Selected edge aggregators  $\mathcal{E} \leftarrow \text{execute Algorithm 2}$ ;
7 Associated nodes  $\mathcal{M}_e, \forall e \in \mathcal{E} \leftarrow \text{execute Algorithm 1}$ ;
8 foreach communication round  $r = 1, 2, \dots, \kappa$  do
9   Send  $w(r-1)$  to each edge aggregator  $e \in \mathcal{E}$ ;
10  foreach edge aggregator  $e \in \mathcal{E}$  in parallel do
11     $(D_e, w_e) \leftarrow \text{EdgeUpdate}(e, \kappa_c, \mathcal{M}_e, w)$ ;
12     $w(r) \leftarrow \frac{\sum_{e \in \mathcal{E}} D_e w_e}{\sum_{e \in \mathcal{E}} D_e}$ ;
13 return  $w(\kappa)$ ;
/* Edge aggregator's process */
14 Function EdgeUpdate( $e, \kappa_c, \mathcal{M}_e, w$ ):
15    $w_e(0) \leftarrow w$ ;
16   foreach edge epoch  $t = 1, 2, \dots, \kappa_c$  do
17     Send  $w_e(t-1)$  to each node  $n \in \mathcal{M}_e$ ;
18     foreach node  $n \in \mathcal{M}_e$  in parallel do
19        $(D_n, w_n) \leftarrow \text{LocalUpdate}(n, e, \kappa_e, w_e)$ ;
20        $D_e \leftarrow \sum_{n \in \mathcal{M}_e} D_n$ ;
21        $w_e(t) \leftarrow \sum_{n \in \mathcal{M}_e} \frac{D_n}{D_e} w_n$ ;
22   return  $D_e$  and  $w_e(\kappa_c)$  to the cloud server;
/* Node's process */
23 Function PreTrain( $n, \kappa_p$ ):
24   Download  $w(0)$  from the cloud server;
25    $(D_n, w_n) \leftarrow \text{LocalUpdate}(n, \kappa_p, w(0))$ ;
26   return  $D_n$  and  $w_n^l$  to the cloud server;
27 Function LocalUpdate( $n, e, \kappa_e, w_e$ ):
28    $w_n(0) \leftarrow w_e$ ;
29    $B \leftarrow \text{split local data } \mathcal{D}_n \text{ into batches}$ ;
30   foreach local epoch  $e = 1, 2, \dots, \kappa_e$  do
31     for batch  $b \in B$  do
32        $w_n(e) \leftarrow w_n(e-1) - \eta \nabla \mathcal{L}(w_n(e-1), b)$ ;
33   return  $D_n = |\mathcal{D}_n|$  and  $w_n(\kappa_e)$  to edge aggregator  $e$ ;

```

V. PERFORMANCE EVALUATION

In this section, we conduct extensive data-driven experiments based on well-known learning tasks and real-world network topologies, to evaluate the performance of ShapeFL.

A. Experimental Setup

Datasets and Models. To evaluate the performance of ShapeFL, we adopt three opened datasets and common CNN

²Here, we assume that the cloud server has the knowledge of the communication costs of the system network.

models: 1) Fashion-MNIST (FMNIST) [26] is a dataset of grayscale fashion product images, which consists of 60,000 training data samples and 10,000 test data samples from 10 categories. We apply the LeNet5 [27] model for classification on it. 2) CIFAR-10 [28] consists of 60,000 32×32 colour images in 10 classes, and we use the MobileNetV2 [29] model for classification. 3) CIFAR-100 has 100 classes of image samples that are similar to CIFAR-10, and we employ the ResNet18 [30] model working on it. To emulate imbalanced local data on distributed computing nodes, we adopt a widely used data partition method in [11]. That is, each training dataset is divided into shards of size 15, and each computing node is distributed with s shards from k classes. Specifically, for the FMNIST and CIFAR-10 dataset, we set $s = 12$ and $k = 4$, while for the CIFAR-100 dataset, we set $s = 100$ and $k = 20$.

Network Topologies. To simulate the communication costs among the cloud server and distributed computing nodes, we consider three network topologies with different geo-locations in the Internet Topology Zoo [31], i.e., Geant2010 (Europe, 37 nodes), Uninet (USA, 42 nodes), and Tinet (Global, 46 nodes). The selected networks have the latitude and longitude information of distributed nodes, which can be used to calculate the distance between nodes via the Haversine formula [32]. Then, we calculate the shortest path distance between nodes using the Dijkstra algorithm. Besides, in each network topology, one additional node, located in Seattle, USA, is added as the cloud aggregator. Generally, the communication cost is reflected by the congestion level posed to the network and the traversed distance of packets. In this paper, we mimic the communication cost via the physical distance between two communicating nodes, multiplied by a coefficient and the volume of the transmitted data packet among them, which has been widely adopted in the literature [33]. In specific, we set $c_{ne} = 0.002 \cdot d_{ne} \cdot S_m$ and $c_{ec} = 0.02 \cdot d_{ec} \cdot S_m$, where d_{ne} (km) is the shortest path distance between node n and edge aggregator e , d_{ec} (km) is the shortest path distance between edge aggregator e and the cloud aggregator, and S_m (GB) is the transmitted data volume of model updates.

Baselines. In our evaluation, we compare the performance of ShapeFL with the following baselines, which can be categorized into:

(1) the typical cloud-based FL methods, where distributed computing nodes commit the model parameters to the cloud server for aggregation, including

- **FedAvg** [11]: the classic FL algorithm.
- **FedProx** [20]: a state-of-the-art federated optimization algorithm that solves the challenges of data imbalance in FL by adding a proximal term to the local objective.

(2) the HFL methods, where distributed computing nodes commit the model parameters to their associated edge aggregator for edge aggregation and then edge aggregators upload the aggregated model parameters to the cloud server for cloud aggregation. The following baselines utilize different approaches for edge aggregator selection and node-edge association, including

- **Cost First**: is a practical edge aggregator selection and node-edge association strategy that can minimize the

per-round communication cost between computing nodes, edge aggregators, and the cloud aggregator by utilizing the CPLEX solver. However, the data distribution of edge aggregators are not taken into consideration.

- **Data First**: associates distributed nodes to the edge aggregator that can make the edge aggregator's data distribution closest to uniform distribution, which has been demonstrated as an effective method to mitigate the imbalanced data issue [34], [35].
- **Random**: associates distributed nodes with edge aggregators randomly.
- **SHARE** [1]: a preliminary version of ShapeFL, which simultaneously reduce the per-round communication cost and the divergence between the data distribution of edge aggregators and the uniform distribution.

Evaluation Metrics. In this paper, we aim to reduce the total communication cost of FL required to reach a target learning accuracy and improve the model convergence accuracy. Therefore, our evaluation mainly focuses on the total communication cost of the model learning to a target accuracy and the final convergence accuracy of the global model measured on the test dataset.

Hyperparameters. For ShapeFL and the baselines, we set batch size $B = 32$ and the learning rate as 0.001. For other specific hyperparameters in ShapeFL, we set the pre-training epoch κ_p of each computing node in Algorithm 1 as 30, and set the maximum iteration T in Algorithm 2 as 30. For the baselines, we have tried to find their best hyperparameter settings for the optimal experimental results.

B. Performance Comparison With Cloud-Based FL

We first compare the performance of ShapeFL with the cloud-based FL baselines, i.e., FedAvg and FedProx, using different learning tasks and network topologies. On the one hand, we investigate the achieved learning accuracy of ShapeFL and baselines after 500 local epochs under different aggregation frequencies (AF, κ_e , κ_c), and the results are shown in Fig. 10. Three major observations can be achieved from the results. First, in most cases, ShapeFL outperforms the baselines significantly. In particular, compared to FedAvg, ShapeFL improves average test accuracy by +3.04% on FMNIST, +17.17% on CIFAR-10, and +10.6% on CIFAR-100. Compared to FedProx, ShapeFL achieves an average of +2.87%, +16.72%, and +10.65% accuracy improvement on FMNIST, CIFAR-10, and CIFAR-100, respectively. Second, increasing the edge aggregation frequency, i.e., decreasing κ_e , contributes to an enhanced learning accuracy. For example, in ResNet18 CIFAR-100, decreasing κ_e from 4 to 2, 1, the test accuracy improves from 26.6% to 33% and 38.3% in average. Third, decreasing the cloud aggregation frequency leads to drastic performance degradation in FedAvg and FedProx, but has little impact on ShapeFL. For example, in MobileNet CIFAR-10 with Uninet, when the cloud aggregation frequency varies from per 4 local epochs to per 18, the test accuracy drops from about 51% to 21% for FedAvg and FedProx, while for ShapeFL, the accuracy maintains about 52% when decreasing the cloud aggregation frequency (i.e., increasing κ_c). The results demonstrate the superiority of ShapeFL in terms of learning accuracy.

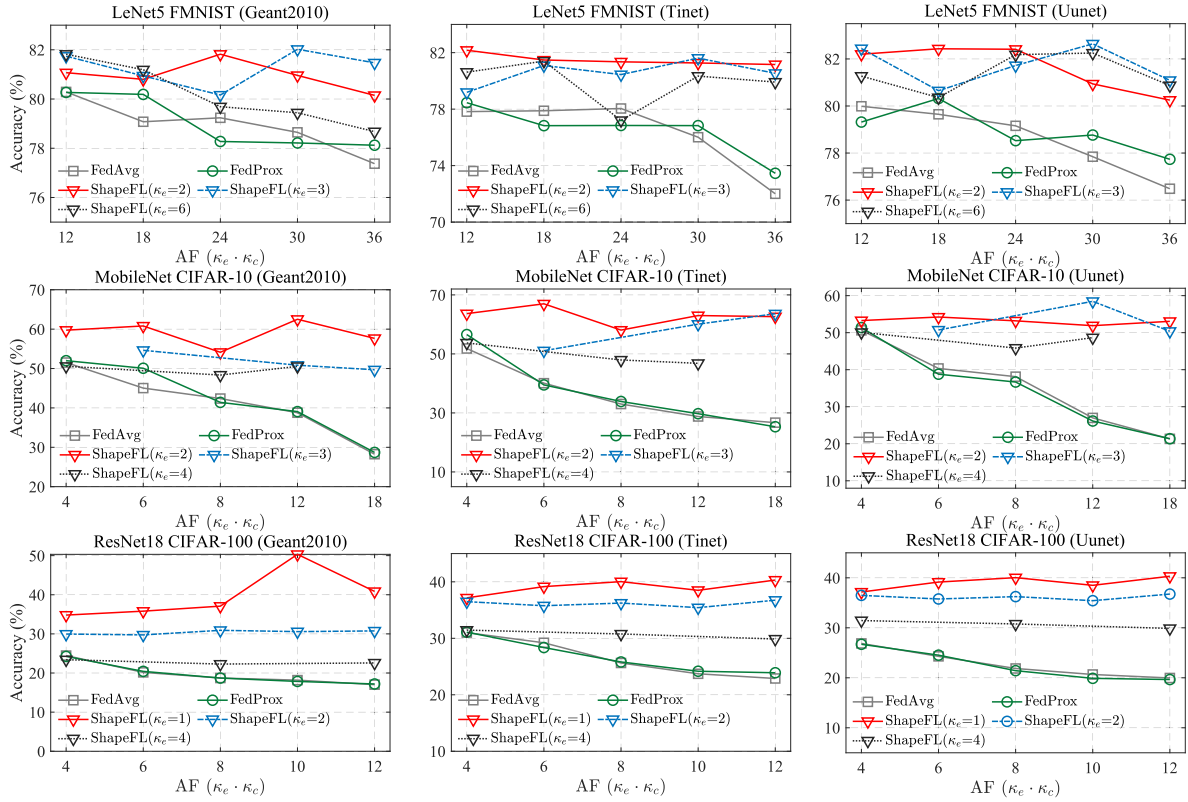
Fig. 10. The achieved learning accuracy after 500 local epochs under varying aggregation frequencies (AF, κ_e , κ_c).

TABLE II

COMMUNICATION COST OF SHAPEFL AND BASELINES REQUIRED TO REACH THE TARGET ACCURACY UNDER VARYING AGGREGATION FREQUENCIES (AF, κ_e , κ_c) AND NETWORK TOPOLOGIES (GEANT2010/TINET/UUNET), WHERE ‘-’ REPRESENTS THE TARGET ACCURACY IS UNACHIEVABLE AND ‘*’ MEANS THE EVALUATION IS NOT PERFORMED IN THIS SETTING

(a) LeNet5 FMNIST					
AF ($\kappa_e \cdot \kappa_c$)	12	18	24	30	36
FedAvg	7.3/14.8/4	5.5/12.9/3.33	5.5/14.8/4	5.5/20.3/4	5.5/-/4.67
FedProx	7.3/12.9/4	5.5/11.1/3.33	5.5/12.9/3.33	5.5/16.6/4	5.5/18.45/4.67
ShapeFL($\kappa_e=2$)	2.24/4.42/1.33	1.73/7.36/1.39	1.2/3.41/1.21	1.3/3.74/1.3	1.43/3.21/2.16
ShapeFL($\kappa_e=3$)	1.97/4.42/0.88	1.34/5.06/1.06	1.52/3.37/0.96	1.1/2.97/1.08	1.2/3.41/1.21
ShapeFL($\kappa_e=6$)	1.04/2.95/0.76	1.12/2.06/0.55	1.18/3.86/0.66	0.85/4.02/0.69	0.9/3.79/0.8
(b) MobileNet CIFAR-10 ($\times 10^3$)					
AF ($\kappa_e \cdot \kappa_c$)	4	6	8	12	18
FedAvg	17/21.7/7.8	15.3/23.4/9.1	17/-/-	-/-/-	-/-/-
FedProx	17/19.9/6.7	14.9/23.4/9.1	17.7/-/-	-/-/-	-/-/-
ShapeFL($\kappa_e=2$)	1.94/2.86/0.89	1.49/2.24/0.88	1.38/2.53/0.72	1.07/1.79/0.66	0.83/1.42/0.72
ShapeFL($\kappa_e=3$)	*	2.08/2.8/1.08	*	1.38/2.02/0.65	0.98/1.79/0.61
ShapeFL($\kappa_e=4$)	3.21/3.38/1.31	*	2.01/2.6/0.79	1.28/2.03/0.79	*
(c) ResNet18 CIFAR-100 ($\times 10^4$)					
AF ($\kappa_e \cdot \kappa_c$)	4	6	8	10	12
FedAvg	8.77/4.35/2.19	10.9/3.26/1.8	-/3.26/2.02	-/2.64/2.24	-/3.26/2.36
FedProx	9.24/4.19/1.96	8.77/3.26/1.85	-/3.1/1.74	-/2.79/1.63	-/2.79/-
ShapeFL($\kappa_e=1$)	1.38/1.11/0.48	1.07/0.94/0.37	0.86/0.85/0.32	0.5/0.59/0.27	0.65/1.1/0.26
ShapeFL($\kappa_e=2$)	1.49/0.9/0.33	0.98/0.7/0.32	0.78/0.67/0.25	0.69/0.59/0.2	0.62/0.56/0.21
ShapeFL($\kappa_e=4$)	1.62/0.85/0.35	*	1.2/0.59/0.22	*	0.81/0.44/0.22

On the other hand, we compare the communication cost of ShapeFL and baselines required to reach the target accuracy of 70%, 40% and 20% for FMNIST, CIFAR-10, and CIFAR-100 respectively. The experimental results under different aggregation frequencies and network topologies are shown in Table II. We can observe that, compared to the cloud-based baselines, ShapeFL can save the communication cost considerably. Besides, we notice that, in most cases, as the aggregation becomes less, i.e., increasing AF, κ_c or κ_e , the communication cost required to reach the target accuracy first decreases and then increases. This is because decreasing the model aggregation frequency decelerates the model con-

vergence, leading to more communication rounds. However, regardless of the aggregation frequency settings, ShapeFL exhibits superior communication efficiency compared to baselines.

C. Performance Comparison With HFL Baselines

Then, we compare the performance of ShapeFL with the HFL baselines using different learning tasks and network topologies. Fig. 11 shows the communication cost and the test accuracy during model training, where κ_e and κ_c are set to 1 and 10, respectively. It can be observed that, the performance of ShapeFL is comparable to SHARE and yields significant

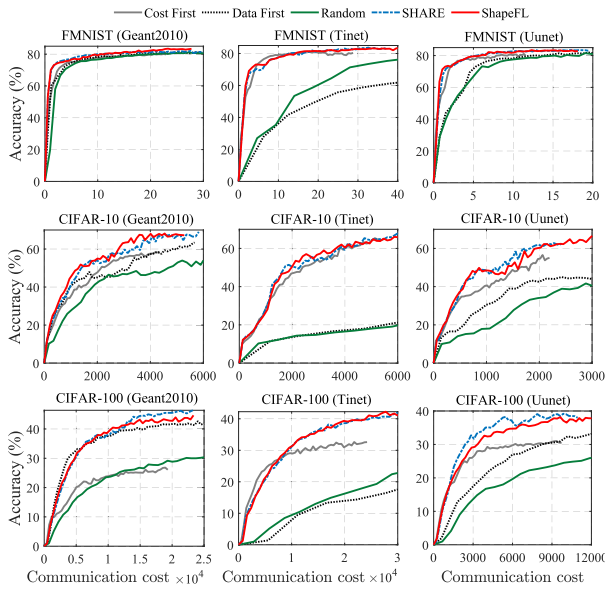


Fig. 11. Accuracy vs. communication cost of ShapeFL and HFL baselines.

performance gains compared to other baseline methods. Particularly, compared to the Cost First, Data First, and Random approaches, ShapeFL takes less communication cost to reach a target learning accuracy. Besides, with the same communication cost, ShapeFL yields higher model accuracy. For example, compared to the Cost First, Data First, and Random baselines, ShapeFL reduces by 40.8%, 48%, and 62.6% communication cost in the LeNet5 FMNIST task under the Geant2010 network topology to achieve 70% accuracy. Furthermore, in the ResNet18 CIFAR-100 task under the Geant2010 network topology, taking the same 1.5×10^4 communication cost, ShapeFL achieves 10%, 20%, and 21% higher test accuracy than the Cost First, Data First, and Random baselines, respectively. The results indicate that ShapeFL is competitive in both communication cost and learning accuracy.

To further present the advantages of ShapeFL in terms of learning accuracy and communication efficiency, we plot the achievable accuracy after 1000 local epochs and the per-round communication cost of ShapeFL and the HFL baselines in Fig. 12. As shown, we notice that the Cost First approach has relatively low per-round communication cost but its achievable accuracy is also relatively low. On the contrary, the per-round communication cost of the Data First method is relatively high, but its achievable accuracy is also higher. It is reasonable since the Cost First method aims to minimize the per-round communication cost while the Data First method focuses on reshaping the imbalanced data distribution that can enhance the learning accuracy. In contrast, ShapeFL achieves relatively high learning accuracy at a relatively low communication cost per round, which indicates that ShapeFL can make a fine trade-off between communication cost and shaping balanced edge data distribution.

D. Robust Experiments

Scalability of ShapeFL. To demonstrate the scalability of ShapeFL, we select a larger-scale network topology *Viatel* with 91 nodes from the Internet Topology Zoo, and evaluate the performance of ShapeFL in different learning tasks. Fig. 15

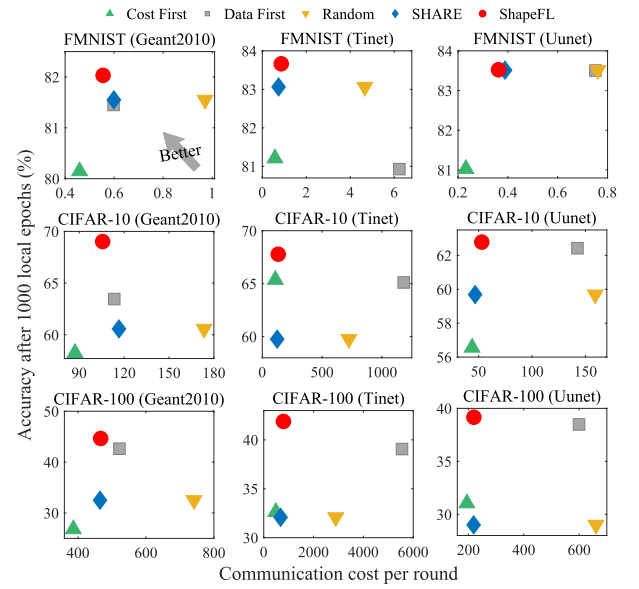


Fig. 12. Accuracy after 1000 local epochs vs. communication cost per round of ShapeFL and HFL baselines.

shows the communication cost and the test accuracy during model training under the Viatel network. It can be observed that ShapeFL also achieves outperformed performance in both model accuracy and communication cost. Compared to the baselines, ShapeFL enables higher model accuracy with less communication cost in large-scale FL systems, which demonstrates the scalability of ShapeFL.

Impact of κ_e . To investigate the impact of edge aggregation frequency on the learning performance, we take the CIFAR-100 task as a demonstration by fixing $\kappa_c = 10$ and varying κ_e from 1 to 3. Fig. 13 shows the achieved model accuracy of ShapeFL and two baselines under different communication costs. It can be observed that as κ_e increases, the performance of the Cost First method decreases, and it performs better when κ_e is relatively small. In contrast, when κ_e increases, the performance of the Data First method becomes better, while the model convergence speed is accelerating and the convergence accuracy is improving. This is because when κ_e is small, model aggregations perform frequently, and hence the per round of communication cost between distributed nodes and edge aggregators dominates in performance, resulting in the Cost First method being more advanced. On the other hand, when the model aggregation is infrequent, data distribution has a larger impact on the model accuracy, and therefore, the Data First method performs better. In addition, we can observe that the performance of ShapeFL increases and then drops as κ_e rises. Although the superiority of ShapeFL can be diluted by Cost First and Data First when κ_e is respective small and large, the performance dominance is obvious when κ_e is in the middle. Overall, ShapeFL outperforms in almost all cases, with accelerated model convergence speed and higher model convergence accuracy.

Impact of κ_c and γ . The impact results of the cloud aggregation frequency κ_c and the trade-off parameter γ are shown in Fig. 14, where we fix $\kappa_e = 2$ and vary κ_c from 5 to 20. It can be observed that the Cost First method converges faster but with lower model convergence accuracy, while the

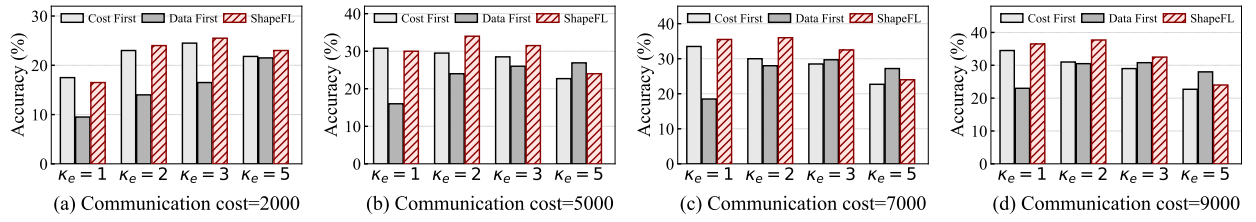
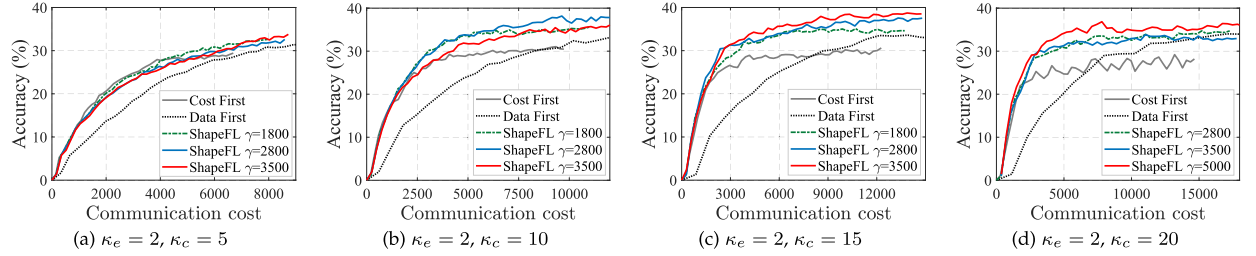
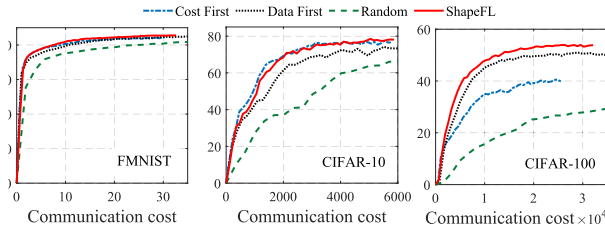
Fig. 13. Impact of κ_e by fixing $\kappa_c = 10$.Fig. 14. Impact of κ_c and γ by fixing $\kappa_e = 2$.

Fig. 15. Accuracy vs. communication cost of ShapeFL and HFL baselines under the Viatel network topology.

Data First method converges slower but with higher model convergence accuracy. It happens since the Cost First method focuses on reducing the per round communication cost, while the Data First method dedicates to shaping balanced data distributions. In contrast, ShapeFL makes a fine trade-off between communication cost and data distribution, achieving a higher model accuracy with less communication cost, and its performance advantage becomes more significant as κ_c grows. Besides, we can see that with the increase of κ_c , enlarging γ makes ShapeFL perform better. For example, $\gamma = 2800$ results in the best performance of ShapeFL when $\kappa_c = 10$, while the optimal performance is achieved in $\gamma = 3500$ and $\gamma = 5000$ when $\kappa_c = 15$ and $\kappa_c = 20$, respectively. This is because when reduces model aggregations by increasing κ_c , the impact of edge data distribution on learning accuracy becomes prominent. Therefore, it is necessary to increase γ to enhance the optimization of edge data distribution in ShapeFL, such that balanced data distributions can be shaped at the edge to guarantee the learning performance of ShapeFL.

VI. IMPLEMENTATION AND DISCUSSION

Implementation. We design and implement two ShapeFL prototypes based on PyTorch. One is an emulation prototype running on the server. It can emulate large-scale computing nodes for model training and communicating, which is used for the previous performance evaluation. Another one prototype is based on a PC as the cloud server and eight NVIDIA Jetson TX2 as distributed computing nodes, which is a hardware implementation. The hardware setup is shown

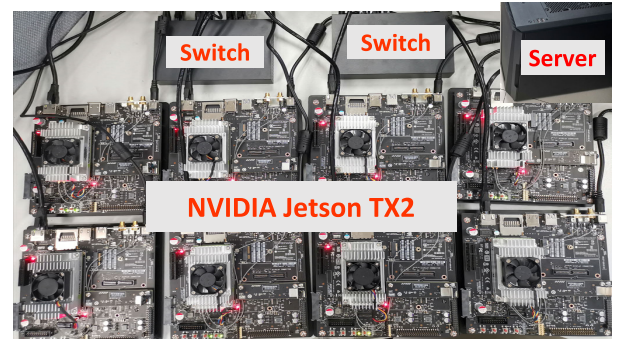


Fig. 16. The hardware implementation of ShapeFL.

in Fig. 16. The PC server is equipped with an Intel Core i9-10900K CPU (3.70GHz) and an NVIDIA's GeForce RTX 3090 graphics card, running Ubuntu 20.04.4, while each TX2 is equipped with ARMv8 CPU and runs Ubuntu 18.04.5. After connecting the PC server and TX2 via Ethernet, ShapeFL can work normally. The hardware implementation demonstrates that our proposed ShapeFL can be readily applied in practical FL systems with an easy-integration manner.

Limitations in Experimental Environment. In the evaluation, we emulated the communication cost between distributed nodes by using real-world network topologies, and conducted simulation experiments to evaluate the learning accuracy and communication efficiency of ShapeFL, where hardware devices and real communication links are not utilized for the following reasons. First, this paper considers a distributed FL system located in a wide area, and therefore, the cost of hardware device deployment can be expensive, especially for large-scale evaluation experiments. Besides, the link conditions between devices are often unstable, especially within a broad area, which also poses challenges for performance evaluation. Although we did not employ hardware devices and real communication link environment for our experiments, we adopted real-world network topologies and set the communication cost between nodes according to practical conditions, the evaluation results of which with large scales are still solid and convincing. Moreover, we have implemented a real system

prototype as shown in Fig. 16, which can support the model training and communicating on hardware nodes, and integrate all functional modules of ShapeFL, being applicable to larger-scale devices.

The Definition of Communication Cost. In this paper, we mainly focus on the data transmission cost over communication links when considering communication cost since it is the most important to our optimization objective. However, it should be noted that the definition and metrics of the communication cost can vary with application scenarios. For instance, hardware resources such as computation and bandwidth of devices can also impact the communication delay, and thus can be taken into account when calculating the communication cost. In such cases, the calculation method of the communication cost between devices (i.e., c_{ne} and c_{ec}) can be adjusted to incorporate various communication cost factors, but the adjustment does not affect the workflow of ShapeFL. ShapeFL works independently of the calculation method of c_{ne} and c_{ec} , since it is designed to select edges and associate clients with edges based on the input information of c_{ne} and c_{ec} . Therefore, the proposed algorithms are readily applied to diverse communication scenarios with different cost definitions.

The Complexity of the Proposed Algorithms. It is straightforward to prove that both the proposed node association algorithm and the edge aggregator selection algorithm are polynomial in complexity, and hence ShapeFL is suitable for large-scale FL systems. Additionally, the node association and edge selection algorithms only need to be performed one time at the resource-substantial cloud server before the FL model training process. Therefore, the time overhead of executing the algorithms can be negligible over the entire hierarchical model training process.

VII. RELATED WORK

Owing to its advanced learning paradigm and privacy-preserving property, FL has attracted considerable research attention in recent years. State-of-the-arts have made efforts to optimize the performance of FL in terms of learning accuracy [19], [21], [36], [37], [38], resource efficiency [39], [40], [41], [42], [43], incentives [44], [45], [46], privacy [47] and security [48], [49], etc. In this section, we mainly summarize existing works dedicated to communication-efficient FL as well as the researches on hierarchical FL and clustered FL which are highly relevant to our work.

A. Communication-Efficient FL

Prior arts that focus on reducing the communication overhead of FL can be roughly divided into three main classes. 1) *Reducing the model update size.* Along this direction, various compression techniques are applied on the model updates to reduce the communicated data volume of each node. For example, QSGD [14], FetchSGD [16], and MUCSC [50] respectively utilized the quantization, sketching, and soft clustering methods to compress the model updates, while Wangni et al. [12] performed sparsification on the model updates such that only a subset of model updates need to be transmitted. 2) *Reducing the number of model update.*

For example, Wang et al. in [17] proposed to dynamically exclude irrelevant individual model updates without committing them to the cloud. Luo et al. [51] reduced FL cost through controlling the number of participating nodes. 3) *Reducing the communication rounds* is another route to reduce the communication overhead. It aims to reduce the number of communication rounds required to reach the target learning accuracy by accelerating model convergence. For example, FedProx [20] and SCAFFOLD [21] are popular FL algorithms with fast convergence under imbalanced data. Unlike those solutions, in this paper, we explore a new direction to reduce the communication cost by migrating expensive model aggregations to the edge. The advantage lies in that frequent communication to the cloud can be avoided, while the learning performance can be guaranteed in the meantime. Moreover, our proposed framework is easy to be integrated with the aforementioned communication optimization methods to collectively improve the communication efficiency of FL.

B. Hierarchical FL

Our method is derived from hierarchical FL, an emerging FL framework that introduces hierarchical model aggregation. Recently, the flexibility and efficiency of hierarchical FL have attracted much attention in the research community. However, the optimization objectives of existing hierarchical FL works are different from our work. Specifically, Liu et al. in [52] proposed a client-edge-cloud hierarchical FL system along with a hierarchical model aggregation algorithm, and Hosseinalipour et al. in [53] proposed a multi-stage hybrid federated learning framework. However, the system architecture of these works is pre-defined, besides, the heterogeneous communication cost and imbalanced data among distributed nodes are ignored in these works. Chai et al. proposed TiFL [54], a Tier-based FL system that divides nodes into tiers according to their training speed and selects participating nodes with similar training speed per round to mitigate the straggler problem. Nevertheless, the optimization objective of TiFL is the training time instead of the communication cost. Lim et al. in [55] addressed the dynamic edge association and resource allocation problems in hierarchical FL by proposing a hierarchical game framework. Likewise, HFEL [56] was also proposed for edge association and resource allocation to minimize the learning cost of hierarchical FL systems. However, both of them mainly consider the network bandwidth while ignoring the imbalanced data characteristics among distributed nodes. For hierarchical FL systems with imbalanced data, Liu et al. in [57] jointly optimized the data distribution distance and learning latency to design the node association and resource allocation schemes. However, they focus on the uplink channel conditions for resource allocation while we concern the heterogeneous communication costs among nodes for cost minimization. To alleviate the accuracy degradation of FL caused by imbalanced data, Duan et al. in [34] and [35] proposed to rebalance the skewed training data of mobile nodes by creating the mediators to reschedule the training of nodes. However, the communication cost is not discussed in this work. Besides, in our previous version work [1], the

data distribution shaping method can expose the privacy of node data distribution. Wang et al. in [58] and [59] aimed to accelerate the model training of FL via cluster topology construction and hierarchical aggregation. The authors in [60] focused on the issues of resource allocation and incentive design in hierarchical FL. Both the optimization objective and focus of them are different from our work. Mhaisen et al. in [61] proposed to assign mobile nodes to edge nodes by minimizing the distribution distance between edge nodes. However, they only take into account the imbalanced data and the communication range constraints of nodes without considering the communication cost between nodes. Additionally, the edge nodes in their proposed hierarchical FL framework are fixed with limited flexibility.

Unlike most of the existing hierarchical FL research that focuses on optimizing the performance of the hierarchical FL framework, we are instead pursuing how to fully exploit its great potential to improve the performance of FL under imbalanced training data. To this end, we highlight the importance of communication efficiency in FL and fully explore the potential of hierarchical FL in reducing the communication cost and improving the learning accuracy of FL through extensive empirical studies. By considering the communication cost among computing nodes, edge aggregators, and the cloud aggregator and shaping balanced edge data distributions simultaneously, we devise efficient edge aggregator selection and node-edge association schemes to minimize the communication cost of HFL systems.

C. Clustered FL

It is worth noting that our work is also related to clustered FL, which also divides the mobile nodes into clusters for efficient learning. The common idea of existing researches on clustered FL is to cluster the nodes with similar data distributions into the same cluster to learn a shared model collaboratively. For example, Ghosh et al. proposed IFCA [62], which assigns nodes to clusters based on the calculated local loss values of the cluster model. Briggs et al. [22] introduced a hierarchical clustering algorithm to group nodes into clusters according to the similarity of their local model updates. Ouyang et al. in [23] applied clustered FL to human activity recognition and utilized alternating optimization approach for clustering. The difference between clustered FL and our proposed hierarchical FL lies in that, after learning, clustered FL acquires multiple models for each cluster, while hierarchical FL just requires one cloud global model. Moreover, clustered FL groups nodes with similar data distributions into the same cluster, whereas we advocate grouping nodes to make the data distribution more balanced per cluster. The inference performance of the models learned by these two methods will be distinct in application. In clustered FL, although the model learned by each cluster can perform well on the nodes falling within that cluster, each learned cluster model is tailored for a specific data distribution. It means that, when the data distribution of mobile nodes changes or newcomer nodes appear, the learned model should be retrained to adapt to new data distributions. In contrast, the cloud global model

learned by our hierarchical FL framework is a general model that can handle any data distribution in principle. Therefore, the learned cloud model can be directly deployed on mobile nodes or newcomer nodes for inference without retraining.

VIII. CONCLUSION AND FUTURE WORK

In this paper, we have fully exploited the underlying potential of HFL framework in terms of communication efficiency and learning accuracy with pilot experiments. Under the HFL framework, we have formulated the communication cost minimization problem and proposed ShapeFL to optimize the total communication cost for model learning. Based on the data-driven insights, in ShapeFL, we have followed two optimization directions, i.e., reducing the per-round communication cost and the required communication rounds by shaping data distribution at edge, and devised two lightweight algorithms to strike a fine trade-off between them. Extensive experiments have been carried out under various real-world network topologies with multiple learning tasks, and the results have demonstrated the efficacy of ShapeFL. For our future work, we will further consider the resource constraints and training time of distributed computing nodes to further enhance the flexibility and efficiency of ShapeFL.

REFERENCES

- [1] Y. Deng et al., "SHARE: Shaping data distribution at edge for communication-efficient hierarchical federated learning," in *Proc. IEEE 41st Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Jul. 2021, pp. 24–34.
- [2] Y. Huang, Y. Zeng, F. Ye, and Y. Yang, "Fair and protected profit sharing for data trading in pervasive edge computing environments," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, Jul. 2020, pp. 1718–1727.
- [3] J. Ren, D. Zhang, S. He, Y. Zhang, and T. Li, "A survey on end-edge-cloud orchestrated network computing paradigms: Transparent computing, mobile edge computing, fog computing, and cloudlet," *ACM Comput. Surv.*, vol. 52, no. 6, pp. 1–36, 2019.
- [4] Y. Zhang, X. Lan, J. Ren, and L. Cai, "Efficient computing resource sharing for mobile edge-cloud computing networks," *IEEE/ACM Trans. Netw.*, vol. 28, no. 3, pp. 1227–1240, Jun. 2020.
- [5] N. Cheng et al., "Space/aerial-assisted computing offloading for IoT applications: A learning-based approach," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 5, pp. 1117–1129, May 2019.
- [6] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proc. IEEE*, vol. 107, no. 8, pp. 1738–1762, Aug. 2019.
- [7] F. Lyu et al., "LeaD: Large-scale edge cache deployment based on spatio-temporal WiFi traffic statistics," *IEEE Trans. Mobile Comput.*, vol. 20, no. 8, pp. 2607–2623, Aug. 2021.
- [8] L. Zeng, X. Chen, Z. Zhou, L. Yang, and J. Zhang, "CoEdge: Cooperative DNN inference with adaptive workload partitioning over heterogeneous edge devices," *IEEE/ACM Trans. Netw.*, vol. 29, no. 2, pp. 595–608, Apr. 2021.
- [9] T. Pasquier, J. Singh, J. Powles, D. Eysers, M. Seltzer, and J. Bacon, "Data provenance to audit compliance with privacy policy in the Internet of Things," *Pers. Ubiquitous Comput.*, vol. 22, no. 2, pp. 333–344, 2018.
- [10] K. R. Sollins, "IoT big data security and privacy versus innovation," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 1628–1635, Apr. 2019.
- [11] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Artif. Intell. Statist.*, 2017, pp. 1273–1282.
- [12] J. Wangni, J. Wang, J. Liu, and T. Zhang, "Gradient sparsification for communication-efficient distributed optimization," in *Proc. Int. Conf. Adv. Neural Inf. Process. Syst.*, 2018, pp. 1299–1309.
- [13] A. Fikri Aji and K. Heafield, "Sparse communication for distributed gradient descent," 2017, *arXiv:1704.05021*.
- [14] D. Alistarh, D. Grubic, J. Li, R. Tomioka, and M. Vojnovic, "QSGD: Communication-efficient SGD via gradient quantization and encoding," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 1709–1720.

- [15] J. Konečný, H. Brendan McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," 2016, *arXiv:1610.05492*.
- [16] D. Rothchild et al., "FetchSGD: Communication-efficient federated learning with sketching," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 8253–8265.
- [17] L. Wang, W. Wang, and B. Li, "CMFL: Mitigating communication overhead for federated learning," in *Proc. IEEE 39th Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Jul. 2019, pp. 954–964.
- [18] S. Ji, W. Jiang, A. Walid, and X. Li, "Dynamic sampling and selective masking for communication-efficient federated learning," 2020, *arXiv:2003.09603*.
- [19] Y. Deng et al., "AUCTION: Automated and quality-aware client selection framework for efficient federated learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 8, pp. 1996–2009, Aug. 2022.
- [20] T. Li, A. Kumar Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," 2018, *arXiv:1812.06127*.
- [21] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "Scaffold: Stochastic controlled averaging for federated learning," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 5132–5143.
- [22] C. Briggs, Z. Fan, and P. Andras, "Federated learning with hierarchical clustering of local updates to improve training on non-IID data," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2020, pp. 1–9.
- [23] X. Ouyang, Z. Xie, J. Zhou, J. Huang, and G. Xing, "ClusterFL: A similarity-aware federated learning system for human activity recognition," in *Proc. 19th Annu. Int. Conf. Mobile Syst., Appl., Services*, Jun. 2021, pp. 54–66.
- [24] M. Pal, T. Tardos, and T. Wexler, "Facility location with nonuniform hard capacities," in *Proc. IEEE FOCS*, Oct. 2001, pp. 329–338.
- [25] X. Lin and S. Wang, "Joint user association and base station switching on/off for green heterogeneous cellular networks," in *Proc. IEEE Int. Conf. Commun. (ICC)*, May 2017, pp. 1–6.
- [26] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," 2017, *arXiv:1708.07747*.
- [27] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
- [28] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," Citeseer, Princeton, NJ, USA, Tech. Rep. TR-2009, 2009.
- [29] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 4510–4520.
- [30] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
- [31] S. Knight, H. X. Nguyen, N. Falkner, R. Bowden, and M. Roughan, "The Internet topology zoo," *IEEE J. Sel. Areas Commun.*, vol. 29, no. 9, pp. 1765–1775, Oct. 2011.
- [32] C. C. Robusto, "The cosine-haversine formula," *Amer. Math. Monthly*, vol. 64, no. 1, pp. 38–40, 1957.
- [33] Q. Qin, K. Poularakis, G. Iosifidis, and L. Tassiulas, "SDN controller placement at the edge: Optimizing delay and overheads," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, Apr. 2018, pp. 684–692.
- [34] M. Duan et al., "Astraea: Self-balancing federated learning for improving classification accuracy of mobile deep learning applications," in *Proc. IEEE 37th Int. Conf. Comput. Design (ICCD)*, Nov. 2019, pp. 246–254.
- [35] M. Duan, D. Liu, X. Chen, R. Liu, Y. Tan, and L. Liang, "Self-balancing federated learning with global imbalanced data in mobile systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 1, pp. 59–71, Jan. 2021.
- [36] F. Lai, X. Zhu, H. V. Madhyastha, and M. Chowdhury, "OORT: Efficient federated learning via guided participant selection," in *Proc. 15th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, 2021, pp. 19–35.
- [37] Q. Wu, X. Chen, Z. Zhou, and J. Zhang, "FedHome: Cloud-edge based personalized federated learning for in-home health monitoring," *IEEE Trans. Mobile Comput.*, vol. 21, no. 8, pp. 2818–2832, Aug. 2022.
- [38] L. Collins, H. Hassani, A. Mokhtari, and S. Shakkottai, "Exploiting shared representations for personalized federated learning," in *Proc. Int. Conf. Mach. Learn. (ICML)*, Jul. 2021, pp. 2089–2099.
- [39] Y. G. Kim and C.-J. Wu, "AutoFL: Enabling heterogeneity-aware energy efficient federated learning," in *Proc. MICRO*, Oct. 2021, pp. 183–198.
- [40] S. Wang et al., "Adaptive federated learning in resource constrained edge computing systems," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 6, pp. 1205–1221, Jun. 2019.
- [41] J. Liu et al., "Adaptive asynchronous federated learning in resource-constrained edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 2, pp. 674–690, Feb. 2023.
- [42] L. Wang, S. Xu, X. Wang, and Q. Zhu, "Addressing class imbalance in federated learning," 2020, *arXiv:2008.06217*.
- [43] C. T. Dinh et al., "Federated learning over wireless networks: Convergence analysis and resource allocation," *IEEE/ACM Trans. Netw.*, vol. 29, no. 1, pp. 398–409, Feb. 2021.
- [44] Y. Deng et al., "Improving federated learning with quality-aware user incentive and auto-weighted model aggregation," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 12, pp. 4515–4529, Dec. 2022.
- [45] J. Pang, J. Yu, R. Zhou, and J. C. S. Lui, "An incentive auction for heterogeneous client selection in federated learning," *IEEE Trans. Mobile Comput.*, vol. 22, no. 10, pp. 5733–5750, Oct. 2023, doi: 10.1109/TMC.2022.3182876.
- [46] Y. Jiao, P. Wang, D. Niyato, B. Lin, and D. I. Kim, "Toward an automated auction framework for wireless federated learning services market," *IEEE Trans. Mobile Comput.*, vol. 20, no. 10, pp. 3034–3048, Oct. 2021.
- [47] K. Wei et al., "User-level privacy-preserving federated learning: Analysis and performance optimization," *IEEE Trans. Mobile Comput.*, vol. 21, no. 9, pp. 3388–3401, Sep. 2022.
- [48] F. Mo, H. Haddadi, K. Katevas, E. Marin, D. Perino, and N. Kourtellis, "PPFL: Privacy-preserving federated learning with trusted execution environments," in *Proc. 19th Annu. Int. Conf. Mobile Syst., Appl., Services*, Jun. 2021, pp. 94–108.
- [49] C. Niu et al., "Billion-scale federated learning on mobile clients: A submodel design with tunable privacy," in *Proc. 26th Annu. Int. Conf. Mobile Comput. Netw.*, 2020, pp. 1–14.
- [50] L. Cui, X. Su, Y. Zhou, and Y. Pan, "Slashing communication traffic in federated learning by transmitting clustered model updates," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 8, pp. 2572–2589, Aug. 2021.
- [51] B. Luo, X. Li, S. Wang, J. Huang, and L. Tassiulas, "Cost-effective federated learning in mobile edge networks," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 12, pp. 3606–3621, Dec. 2021.
- [52] L. Liu, J. Zhang, S. H. Song, and K. B. Letaief, "Client-edge-cloud hierarchical federated learning," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Jun. 2020, pp. 1–6.
- [53] S. Hosseinalipour et al., "Multi-stage hybrid federated learning over large-scale D2D-enabled fog networks," *IEEE/ACM Trans. Netw.*, vol. 30, no. 4, pp. 1569–1584, Aug. 2022.
- [54] Z. Chai et al., "TiFL: A tier-based federated learning system," in *Proc. 29th Int. Symp. High-Perform. Parallel Distrib. Comput.*, Jun. 2020, pp. 125–136.
- [55] W. Y. B. Lim, J. S. Ng, Z. Xiong, D. Niyato, C. Miao, and D. I. Kim, "Dynamic edge association and resource allocation in self-organizing hierarchical federated learning networks," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 12, pp. 3640–3653, Dec. 2021.
- [56] S. Luo, X. Chen, Q. Wu, Z. Zhou, and S. Yu, "HFEL: Joint edge association and resource allocation for cost-efficient hierarchical federated edge learning," *IEEE Trans. Wireless Commun.*, vol. 19, no. 10, pp. 6535–6548, Oct. 2020.
- [57] S. Liu, G. Yu, X. Chen, and M. Bennis, "Joint user association and resource allocation for wireless hierarchical federated learning with IID and non-IID data," *IEEE Trans. Wireless Commun.*, vol. 21, no. 10, pp. 7852–7866, Oct. 2022.
- [58] Z. Wang, H. Xu, J. Liu, H. Huang, C. Qiao, and Y. Zhao, "Resource-efficient federated learning with hierarchical aggregation in edge computing," in *Proc. IEEE Conf. Comput. Commun.*, 2021, pp. 1–10.
- [59] Z. Wang, H. Xu, J. Liu, Y. Xu, H. Huang, and Y. Zhao, "Accelerating federated learning with cluster construction and hierarchical aggregation," *IEEE Trans. Mobile Comput.*, vol. 22, no. 7, pp. 3805–3822, Jul. 2023, doi: 10.1109/TMC.2022.3147792.
- [60] W. Y. B. Lim et al., "Decentralized edge intelligence: A dynamic resource allocation framework for hierarchical federated learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 3, pp. 536–550, Mar. 2022.
- [61] N. Mhaisen, A. A. Abdellatif, A. Mohamed, A. Erbad, and M. Guizani, "Optimal user-edge assignment in hierarchical federated learning based on statistical properties and network topology constraints," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 1, pp. 55–66, Jan. 2022.

- [62] A. Ghosh, J. Chung, D. Yin, and K. Ramchandran, "An efficient framework for clustered federated learning," in *Proc. NeurIPS*, 2020, pp. 19586–19597.



Yongheng Deng received the Ph.D. degree from the Department of Computer Science and Technology, Tsinghua University, Beijing, China, in 2024. She is currently a Post-Doctoral Researcher with the Department of Computer Science and Technology, Tsinghua University. Her research interests include federated learning, edge intelligence, distributed systems, and edge computing.



Feng Lyu (Senior Member, IEEE) received the Ph.D. degree from the Department of Computer Science and Engineering, Shanghai Jiao Tong University, Shanghai, China, in 2018. From 2018 to 2020, he was a Post-Doctoral Fellow with the Broadband Communications Research Group, Department of Electrical and Computer Engineering, University of Waterloo, Canada. He is currently a Full Professor with the School of Computer Science and Engineering, Central South University, Changsha, China. He has published over 100 scientific papers

in leading journals and top conferences, including Proceedings of the IEEE, IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, IEEE/ACM TRANSACTIONS ON NETWORKING, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, IEEE INFOCOM, *The VLDB Journal*, and ACM SenSys. His research interests include mobile networks, beyond 5G networks, big data measurement and application design, and cloud/edge computing. His research results have been highly visible, with nine publications being ranked as ESI Highly Cited Papers. He was a recipient of the Best Paper Award from IEEE ICC 2019, IEEE/CAA JOURNAL OF AUTOMATICA SINICA "Norbert Wiener Review Award" in 2020, the Outstanding Paper Award from the *Chinese Journal on Internet of Things* in 2021, the IEEE Technical Committee on Hyper-Intelligence (IEEE HITC) 2023 Early Career Researcher, and the IEEE ComSoc Distinguished Lecturer from 2024 to 2025. He currently serves as an Associate Editors for IEEE INTERNET OF THINGS JOURNAL, IEEE SYSTEMS JOURNAL, and *Peer-to-Peer Networking and Applications*, and serves as a TPC member for many international conferences.



Tengxi Xia (Graduate Student Member, IEEE) received the B.S. degree from the Harbin University of Science and Technology, Harbin, China, in 2019. He is currently pursuing the Ph.D. degree with the Department of Computer Science and Technology, Tsinghua University, Beijing, China. His research interests include split learning, federated learning, edge computing, and distributed systems.



Yuezhi Zhou (Senior Member, IEEE) received the Ph.D. degree in computer science and technology from Tsinghua University, Beijing, China, in 2004. He was a Visiting Scientist with the Department of Computer Science, Carnegie Mellon University, in 2005. He is currently a Full Professor with the Department of Computer Science and Technology, Tsinghua University. He has authored or coauthored over 100 technical papers in international journals or conferences. His research interests include pervasive computing, distributed systems, and mobile

networks. He received the Best Paper Award from the IEEE International Conference on Advanced Information Networking and Applications (AINA) in 2007 and the IEEE International Conference on High Performance Computing and Communications (HPCC) in 2014.



Yaoyue Zhang (Senior Member, IEEE) received the B.Sc. degree from the Northwest Institute of Telecommunication Engineering, Xi'an, China, in 1982, and the Ph.D. degree in computer networking from Tohoku University, Sendai, Japan, in 1989. He is currently a Professor with the School of Computer Science and Engineering, Central South University, China, and a Professor with the Department of Computer Science and Technology, Tsinghua University, China. He has published over 200 technical papers in international journals and conferences, as well as nine monographs and textbooks. His research interests include computer networking, operating systems, ubiquitous/pervasive computing, transparent computing, and big data. He is a fellow of the Chinese Academy of Engineering, China, and the Editor-in-Chief of the *Chinese Journal of Electronics*.



Ju Ren (Senior Member, IEEE) received the B.Sc., M.Sc., and Ph.D. degrees in computer science from Central South University, China, in 2009, 2012, and 2016, respectively. He is currently an Associate Professor with the Department of Computer Science and Technology, Tsinghua University, China. His research interests include the Internet of Things, edge computing, operating systems, and as well as security and privacy. He received many Best Paper Awards from IEEE flagship conferences, including IEEE ICC 2019 and IEEE HPCC 2019, the IEEE

TCSC Early Career Researcher Award in 2019, and the IEEE ComSoc Asia-Pacific Best Young Researcher Award in 2021. He also served as the General Co-Chair for IEEE BigDataSE 2020, the TPC Co-Chair for IEEE BigDataSE 2019, the Poster Co-Chair for IEEE MASS 2018, and the Track Co-Chair for IEEE/CIC ICC 2019, IEEE I-SPAN 2018, and VTC 2017 Fall. He is an active reviewer for over 20 international journals. He also serves as an Associate Editor for IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY and *Peer-to-Peer Networking and Applications*. He was recognized as a Highly Cited Researcher by Clarivate from 2020 to 2022.



Yuanyuan Yang (Life Fellow, IEEE) received the B.Eng. and M.S. degrees in computer science and engineering from Tsinghua University, Beijing, China, and the M.S.E. and Ph.D. degrees in computer science from Johns Hopkins University, Baltimore, MD, USA. She is currently a SUNY Distinguished Professor of computer engineering and computer science with Stony Brook University, Stony Brook, NY, USA. She is also on leave with the National Science Foundation as the Program Director. She has published more than 460 papers

in major journals and refereed conference proceedings and holds seven U.S. patents in these areas. Her research interests include edge computing, data center networks, cloud computing, and wireless networks. She is currently the Editor-in-Chief for IEEE TRANSACTIONS ON CLOUD COMPUTING and an Associate Editor for IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS and *ACM Computing Surveys*. She has served as the Associate Editor-in-Chief for IEEE TRANSACTIONS ON CLOUD COMPUTING, the Associate Editor-in-Chief and an Associate Editor of IEEE TRANSACTIONS ON COMPUTERS, and an Associate Editor of IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS. She has also served as the general chair, program chair, or vice chair for several major conferences, and a program committee member for numerous conferences.